

Objections

Run phase is where your test actually runs, but UVM needs to know when work is still in flight

Raise, drop, and the total count

- In run phase you call raise objection on the phase object
- When your task finishes, call drop objection to release your hold
- Multiple components can raise at once, test, sequence, env
- Forget to drop and run hangs with objections stuck above zero
- Drop too early and your test ends before sequences or drivers finish
- The rule is simple: whoever raises should drop

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Objection raise/drop

INTERACTIVE TOOL

Objection raise/drop

See **who holds the run phase open**: raise keeps time alive; drop until the count hits zero ends run. Starter: test has raised — run stays open.

Starter example: test has raise_objection — total count 1, run stays open.

Load starter example

Challenges 0 / 22 cleared

Quiz: raise: raise_objection in run_phase...

☐ keeps the run phase open while work continues

☐ deletes the env hierarchy

☐ skips connect_phase

☐ forces \$finish immediately

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

raise

Keep run (and sim time) alive while work continues.

drop

Release your hold; when total hits 0, run can end.

Who

Tests, sequences, agents... anyone in run may hold.

Balance

Raise without drop → hang; drop too soon → short test.

Lab

SCENARIO

ACTOR

Objections lab starter

learn_uvm2017 — uvm objections

Real UVM literacy

```
wsl - module10-uvvm-objections/examples/objections-sketch

# Track A - real Unix
# real shell session (Track A - UVM objections)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module10-uvvm-objections

$ ls examples/objections-sketch/
holds.txt

$ cat examples/objections-sketch/holds.txt
UVM objections sketch (literacy)

Purpose: keep run_phase open while async work (sequences, forks) runs
        run ends when total objection count hits 0

Pattern in run_phase task:
task run_phase(uvm_phase phase);
    phase.raise_objection(this);    // hold run open
    // start sequences, fork tasks, wait for completion
    my_seq.start(sequencer);
    // or: fork ... join
    phase.drop_objection(this);    // release hold
endtask

Rules:
raise increments total count; drop decrements
total > 0 → run stays open, sim time advances
total == 0 → run can end → check_phase → report_phase
whoever raises must drop (balanced)
multiple components can hold simultaneously - all must drop

Common actors:
test (uvm_test) - often raises around top-level stimulus
env / virtual sequences - may raise for long-running traffic
sequences - may raise/drop inside body for sub-flows

Common mistakes:
raise without drop → sim hangs in run (count stuck > 0)
drop before sequences finish → test ends too early
drop in wrong phase (objections are phase-specific)
double raise without double drop

Tie-in with sequence flow:
raise BEFORE seq.start so run does not end mid-sequence
drop AFTER sequences / forks complete

$ grep -n "raise_objection\|drop_objection"
.../Learn_uvm2017_sv_verilator/module8/tests/uvvm_tests/test_utilities_uvm.sv
253:     phase.raise_objection(this);
259:     phase.drop_objection(this);
```

Real shell — objections sketch

Pitfalls to watch

- Do not raise in build and forget run phase, that is the wrong phase for time advancement
- Do not drop before your sequences finish or run ends too soon
- Do not assume one component's drop clears everyone, each holder must drop
- Double raises need double drops
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load multi and explain why one drop is not enough
- On real UVM, sketch a run phase task with raise, sequence start, and drop
- When you are ready, take the short quiz, then continue to plusargs in the next module